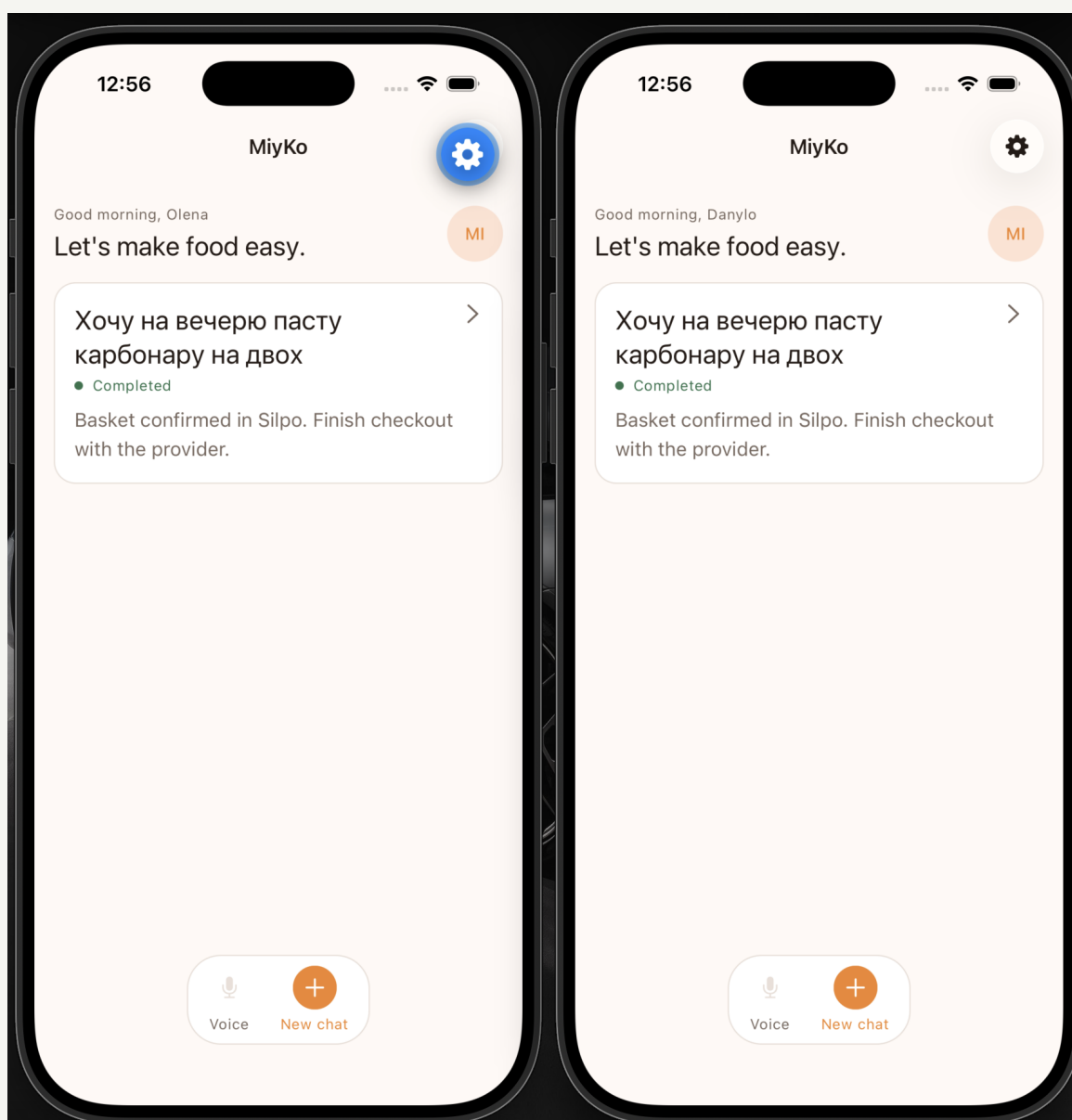


Case Study:

MiyKo

A shared shopping workflow



01

One shared shopping process

MiyKo lets a group plan one grocery order, approve member requests and prepare a store basket. I designed it around a durable LangGraph workflow, with separate owners for permissions, order state, memory and store data.

The problem

A shared order can take hours and involve people with different roles. One person starts a plan, another adds items, and an owner approves a request later. The system needs to keep that order together, check who can act and use current store data when preparing the basket.

Parts of the system

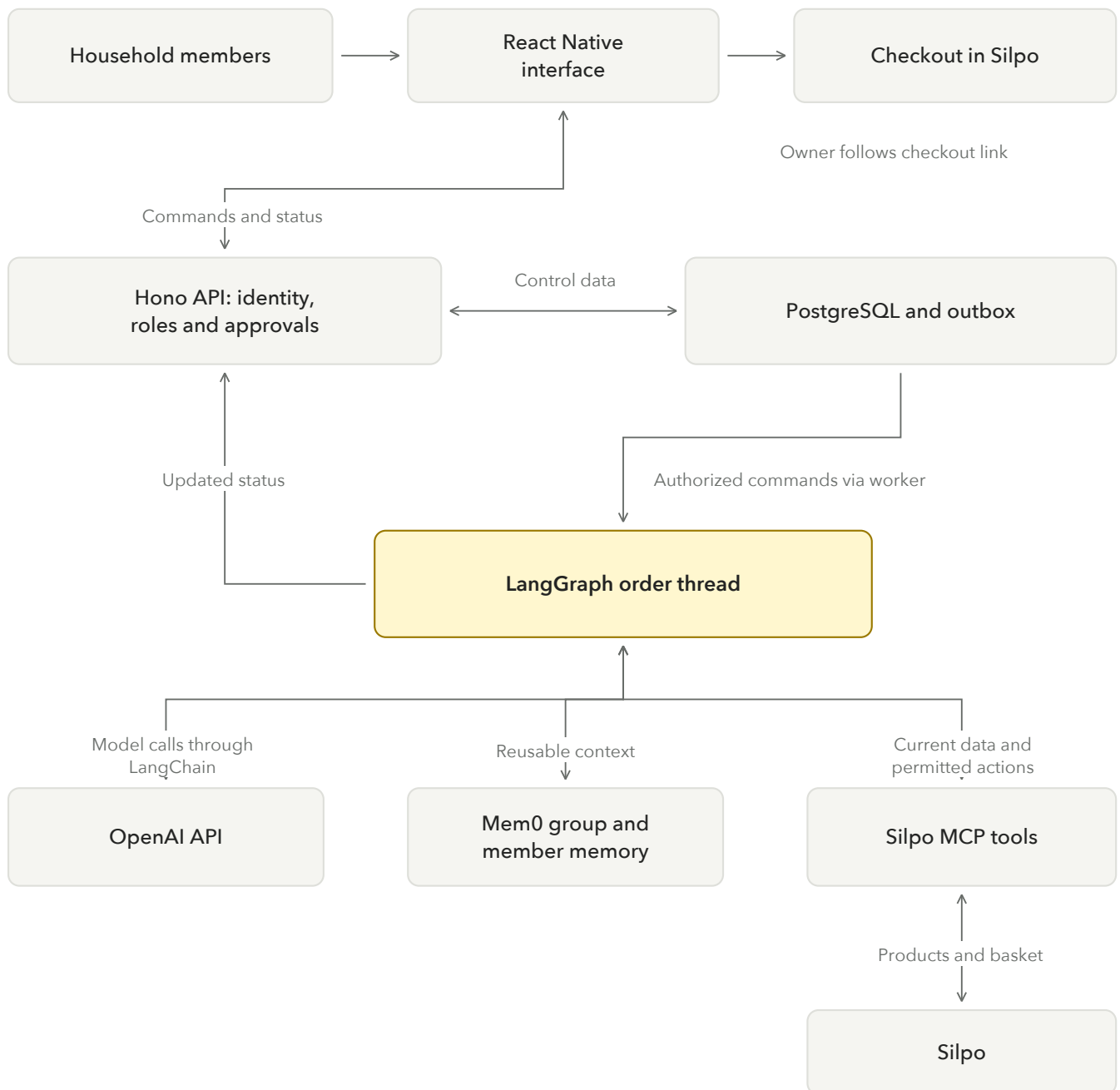
- **React Native app:** the interface for plans, member requests, approvals and basket status.
- **Hono API and PostgreSQL:** authentication, group membership, roles, provider connections, approval records and command delivery.
- **LangGraph:** the active order, saved workflow state and pause/resume steps.
- **LangChain and OpenAI API:** model calls and tool integration within the workflow.
- **Mem0:** reusable group and member preferences across orders.
- **Silpo MCP:** current products, prices, basket operations and checkout links.

The model proposes steps inside the workflow. The API checks the authenticated member before protected actions. An OpenAI API key authenticates model calls; it does not grant permission to change the household's basket. Store access uses a separate provider connection.

React Native displays the process and sends commands. The shared order continues in LangGraph when a member leaves the screen or returns later.

02

System architecture



The API checks authority, LangGraph runs the order, and Silpo owns the basket and purchase.

LangChain connects model calls and tools inside the graph. LangGraph owns the process around those calls: which state is active, where to wait and how to continue. LangSmith supplies development and tracing tools around the workflow.

03

Data ownership

Information	System of record	Boundary
Members, groups, roles and provider bindings	PostgreSQL control plane	Who belongs to the group and may act
Workflow references, approval decisions, audit and outbox	PostgreSQL control plane	Accepted commands and their authorized actors
Active order and pause/resume state	LangGraph thread	The current process and its continuation
Reusable group and member preferences	Mem0	Context for suggestions across orders
Products, prices, basket, fulfillment and checkout	Silpo	Current commercial data and the final transaction

Each store answers a different question:

- **PostgreSQL:** who made the request, and are they allowed to make it?
- **LangGraph:** what is happening in this order, and what happens next?
- **Mem0:** which preferences are useful for this group or member?
- **Silpo:** which products are available, and what is in the real basket?

PostgreSQL stores workflow references, not a second set of LangGraph checkpoints. It also avoids duplicating the store catalog and basket. That keeps order progress and current prices attached to their original owners.

Store connection and credentials

The owner connects Silpo through OAuth with PKCE. MiyKo stores the provider credential encrypted on the backend and binds the connection to the household. Members use that connection through authorized workflow actions without sharing the owner's store login.

When a step needs Silpo MCP, the API supplies the provider token as runtime-only context. It stays outside the mobile app, LangGraph checkpoints and Mem0. Workflow state can therefore retain the order without retaining the store credential in its history.

Provider operations stay behind the MCP integration. The workflow discovers the available tools, reads current product data and applies authorized basket changes. This keeps store-specific requests separate from the shared planning and approval rules.

04

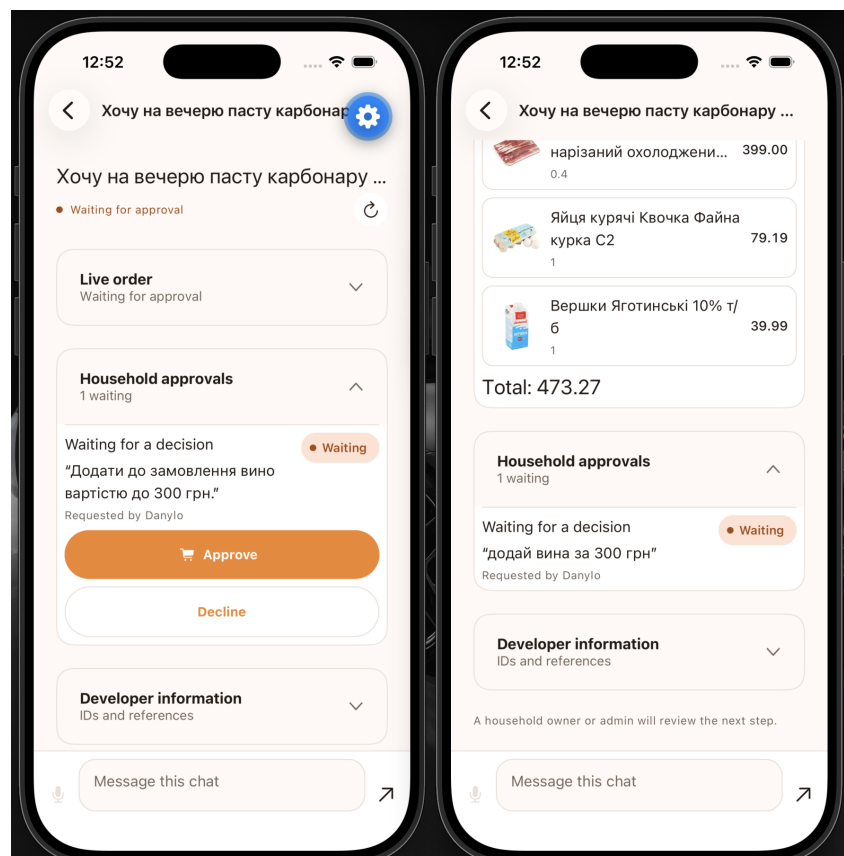
Roles and approvals

Each member has a separate authenticated session. Household membership connects that person to the shared order and its provider connection.

Role	Responsibility in the shared process
Owner or admin	Approve requests and perform provider actions
Editor	Contribute to the shared plan
Viewer	Submit a request that requires approval before affecting the real basket

A protected request stays in the same workflow while it waits for an owner or admin. The API records the decision, and LangGraph resumes that order. A declined request remains declined even if command delivery is retried.

The interface exposes the same states: who is acting, which request is waiting, whether the basket is prepared and which action is available next. These states come from the shared process rather than a separate interpretation in each member's chat.

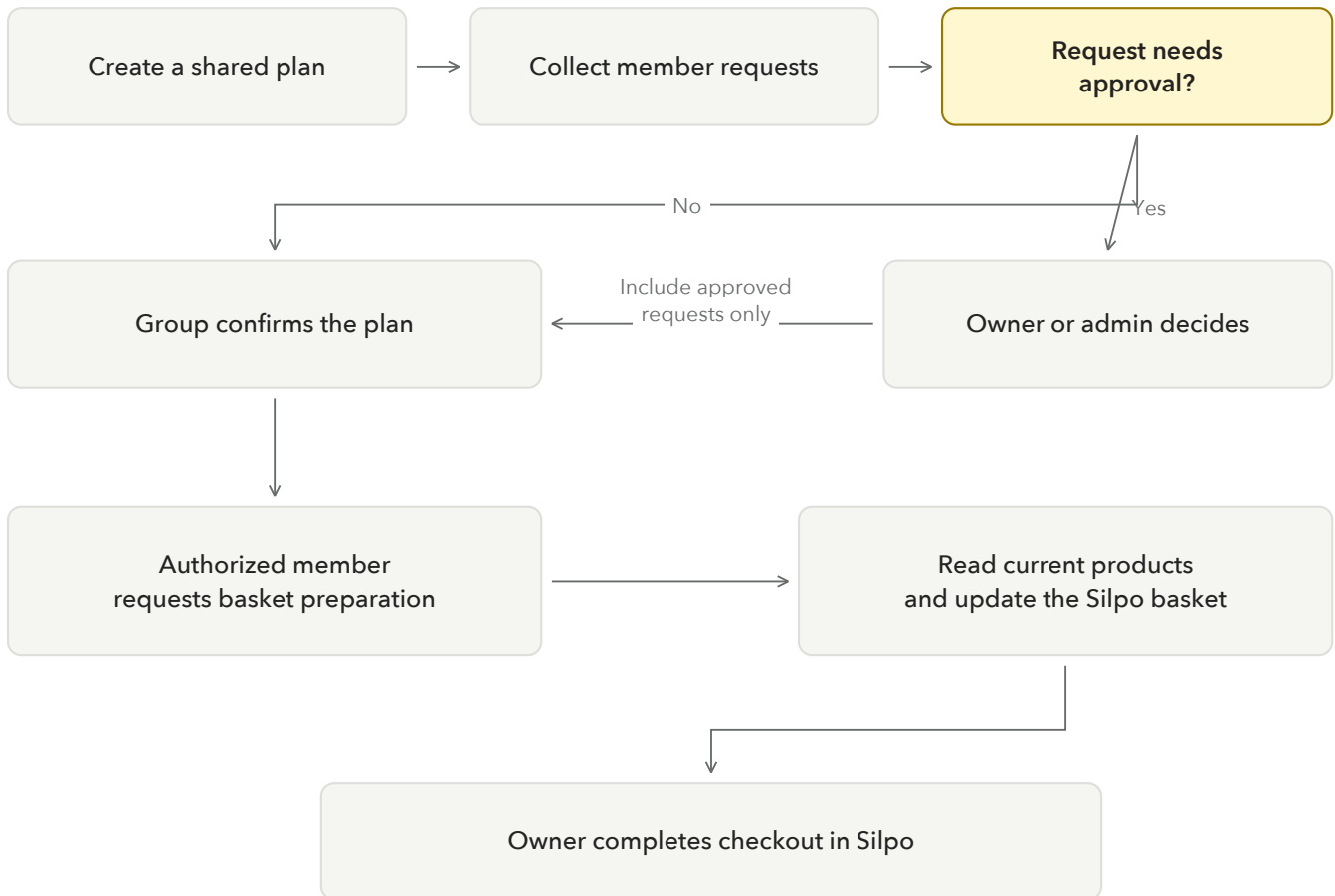


The request and approval belong to the same order, even when different members act at different times.

05

From plan to checkout

The owner connects Silpo before the group prepares a basket. They can also import a summary of recent receipts to give the first plan useful context.

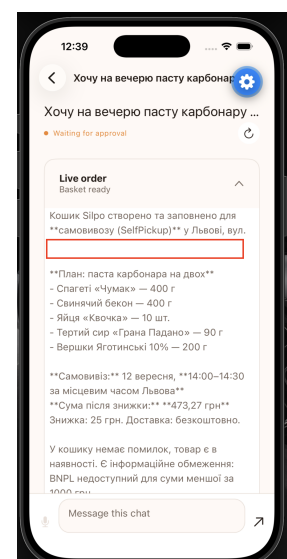


Request approval, plan confirmation and basket preparation are separate steps.

The important rules are:

- Planning and member suggestions do not change the store basket.
- Approving one request does not execute the whole plan.
- Basket preparation requires an authorized member and current provider data.
- A prepared basket is ready for checkout; it is not a confirmed purchase.

Silpo MCP resolves products and updates the provider basket. Names, images, prices and totals come from the store. It returns a checkout link, and the owner finishes the purchase in Silpo.



The app shows the prepared basket. Final checkout stays with Silpo.

Workflow, retries and memory

LangGraph state

Each order has one workflow UUID, reused as its LangGraph thread ID. Member requests and approval decisions resume that thread instead of creating new conversations.

The **step-order** graph covers planning, member changes, approvals, provider actions and completion. Saved workflow state keeps the process at the right step while it waits. When a tool returns a result, that result becomes part of the order state and the next status shown in the app.

Command delivery and idempotency

API command > PostgreSQL transaction and outbox > worker > LangGraph thread

The API records an outbox event alongside the control-plane change. A worker then delivers the command and retries temporary delivery failures.

Two identifiers serve different purposes:

- **Workflow UUID:** directs actions to the same order over its lifetime.
- **Idempotency key:** identifies a command when delivery is retried, so another delivery attempt is not treated as a new action.

The outbox handles the API-to-workflow handoff. Store operations are a separate step and still require permission checks and current provider data. Retrying a command does not change an owner's decision or make an old price authoritative.

Mem0 memory

Mem0 stores reusable context separately from the active order:

- group preferences shared across orders;
- personal context for individual members;
- useful purchase patterns from imported receipt summaries.

Memories can be added, updated or removed. "We usually buy food for two days" can be reusable context. "Add ice cream tonight" belongs to the current LangGraph thread. Today's price belongs to Silpo.

The workflow retrieves relevant memory to improve suggestions. Memory does not supply a member's role, approve a request or replace a current store lookup.

Summary

MiyKo uses LangGraph to keep one order across people and time. The API owns permissions, Mem0 supplies preferences, and Silpo owns products and checkout. Stable workflow IDs, command idempotency and explicit approvals let the group continue the same process from a first suggestion to a prepared basket.